

ICML 2022 | 梯度稀疏化本想省时间，结果压缩自己成了新瓶颈

Rice University 与 ThirdAI 提出 DRAGONN：用随机哈希替换精确前缀和，把压缩开销砍掉最多 70%，端到端训练最快提速 3.52×

在多卡分布式训练里，真正拖慢速度的往往不是算力，而是每一步都要在 GPU 之间同步梯度。梯度稀疏化（gradient sparsification，简称 GS）本来是为解决这个通信瓶颈而生的：既然大部分梯度都很小，那就只传其中最大的一小部分，通信量自然大幅下降。这个思路在很多论文里都成立。

但这篇 ICML 2022 论文观察到一个让人有点意外的现象：当稀疏化真正跑起来，节省下来的通信时间常常被另一项开销吃回去，那就是「压缩」本身的耗时。为了挑出要传的那一小撮梯度，算法需要在 GPU 上做一系列并行操作，而这些操作在大张量上并不便宜。换句话说，稀疏化解决了通信问题，却又制造了一个新的压缩瓶颈。

论文的核心主张很直接：既然稀疏化只需要「近似」的 top-k 梯度，就不应该为一个「精确」的选择算法买单。作者提出 DRAGONN（Distributed Randomized Approximate Gradients of Neural Networks），用直接的随机哈希替换掉现有方法里那套精确的并行前缀和（parallel prefix sum），把压缩开销压到接近理论下界，从而让梯度稀疏化真正在分布式训练里划算起来。

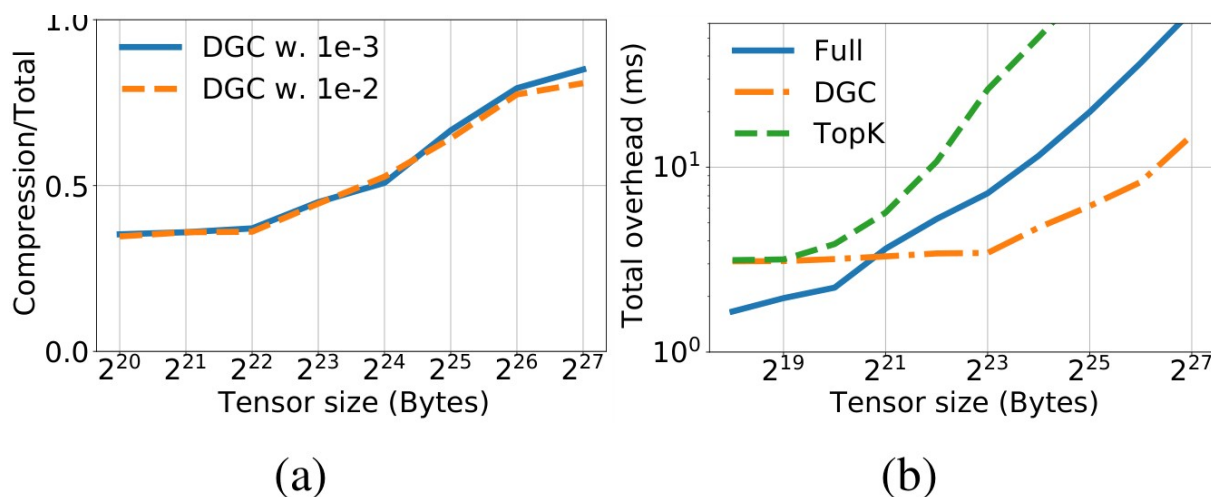


图 1 压缩开销随张量变大而失控。左图：随着张量尺寸增大，压缩时间占单步总开销的比例从约 35% 一路升到 90% 附近；右图：当张量超过约 2^{24} 字节（约 16 MB）后，DGC 与 TopK 的总开销反而超过了完整同步（Full），压缩不再是净收益。

图 1 把问题量化了出来。左图显示，张量越大，压缩时间在单步总耗时里占的比重越高，最终逼近 90%；右图更直接：一旦张量超过约 16 MB，主流稀疏化方法（DGC、TopK）的总开销

甚至高于什么都不压、直接完整同步（Full）的方案。对小张量而言，稀疏化同样可能得不偿失。也就是说，压缩开销已经从「配角」变成了整个流程里最大的那块成本。

论文与代码地址：
论文 <https://proceedings.mlr.press/v162/wang22aj.html>
代码 <https://github.com/zhuangwang93/dragonn>

问题出在「用精确算法做近似的事」

要理解 DRAGONN 的改动，先要看清现有方法慢在哪。当前最有代表性的稀疏化方法是 Deep Gradient Compression（DGC）：它先估一个阈值，把绝对值超过阈值的梯度当作要传的「大梯度」。这一步本身就是近似的（阈值是估出来的）。真正昂贵的是下一步：为了把这些大梯度无冲突地写进一块紧凑内存，DGC 依赖并行前缀和来计算每个非零元素该放到哪个位置。

并行前缀和是一个「精确」算法：它要构建一棵平衡二叉树，分 $O(\log d)$ 个串行步骤扫描整个张量，反复读写内存。论文测算，这套流程带来的内存访问量大约是理论下界（ d 次比较）的 7×。作者称之为「精确 - 近似的错配」：明明只需要一份近似的 top-k 梯度，却动了一个昂贵的精确定位算法去安放它们，这多出来的开销纯属浪费。

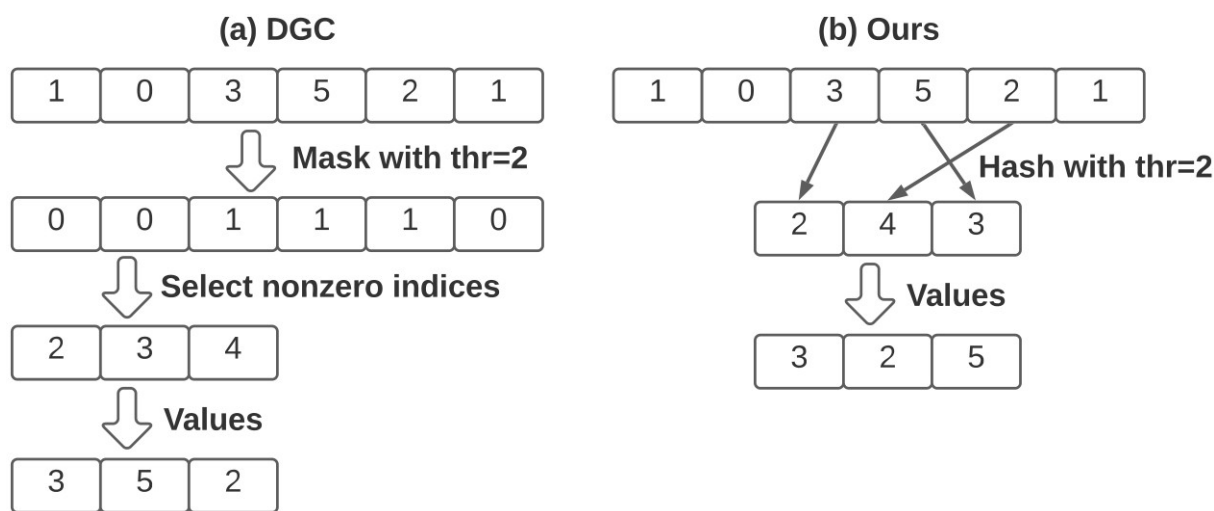


图 2 DGC 与 DRAGONN 的对比。左（DGC）：先用阈值做掩码得到 0/1 序列，再「选出非零索引」（这一步需要前缀和），最后取值，是一条有顺序依赖的串行链路；右（DRAGONN/Ours）：对每个超过阈值的梯度，直接把它的索引哈希到内存槽位并写入，跳过了非零索引选择这一整步。

图 2 把两条路径并排放在一起。左边 DGC 的三步——掩码、选出非零索引、取值——环环相扣，中间那步「选出非零索引」正是前缀和的用武之地，也是顺序依赖的来源。右边 DRAGONN 直接把「哪些位置非零」这个问题绕开了：对每个过阈值的梯度，用一个哈希函数算出它在紧凑缓冲区里的落点，然后写进去，不需要事先知道前面有多少个非零元素。

DRAGONN：把顺序依赖的选择变成可并行的哈希

DRAGONN 的做法可以用一句话概括：预先按压缩率分配一小块内存缓冲区，对每个绝对值超过阈值 t 的梯度 $G[i]$ ，计算 $j = h(i)$ 并直接写入 $I[j] = i$ 。如果两个索引哈希到同一个槽位，后写的直接覆盖先写的；留空的槽位（标记为 -1）在解码时映射为零。整个过程只需要 d 次比较加上 l 次哈希操作，非常接近「 d 次比较」这个理论下界。

为什么这样就快了？关键在于 GPU 上的内存写入是原子操作，因此成千上万个线程可以同时哈希、同时写入，彼此之间没有任何顺序依赖。前缀和那种「必须先算出前面的累加值，才能确定当前位置」的串行链条被彻底拆掉了，压缩从一个串行、依赖繁重的步骤，变成了一个几乎「令人尴尬地并行」（embarrassingly parallel）的步骤。代价是哈希冲突会丢掉极少数梯度，但由于本来就只需要近似 top-k，加上误差反馈机制，这点损失并不影响收敛。

论文还补了两个让它在真实系统里跑得好的工程优化。其一是「效率感知的张量选择」（efficiency-aware tensor selection）：只有当一个张量的压缩时间小于它能省下的通信时间时，才对它做压缩，否则直接完整同步——这正好对应图 1 里小张量压缩不划算的情形。其二是「稀疏解码」（sparse decoding）：把从其他 worker 收到的 K 份压缩张量批量成一次解码，使解码开销几乎不随 GPU 数量增长。

端到端的效果：越多卡，优势越明显

论文在四个模型上做了评测，覆盖视觉与推荐两类负载：ResNet50 在 ImageNet-1K 上训练，ViT 与 MLP-Mixer 在 ImageNet-21k 预训练后于 Cifar10 上微调，以及一个在 Wiki10-31K 上的极端多标签分类（XML）模型。测试平台是 2 台机器共 16 张 Nvidia Tesla V100-32GB，通过 25Gbps 网络互联，软件栈为 PyTorch 1.8.0 + Horovod + NCCL，并对所有稀疏化方法统一使用 memory-momentum 误差反馈以保持精度。

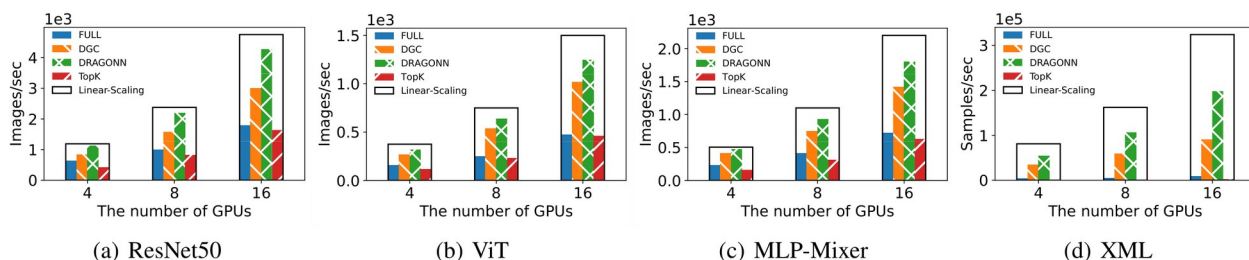


图 3 四个模型在 4/8/16 张 GPU 下的训练吞吐（Images/sec 或 Samples/sec）。黑色空心框表示理想的线性扩展（Linear-Scaling），绿色 DRAGONN 在所有设置下都最接近这个上界，且随 GPU 数量增加，与其它方法的差距进一步拉大。

图 3 是核心结果：绿色的 DRAGONN 在四个模型、三种 GPU 规模下都最靠近黑框代表的线性扩展上界，明显高于 DGC、TopK 和完整同步。更值得注意的是趋势——GPU 从 4 张增加到 16 张时，DRAGONN 相对 DGC 的领先幅度不是缩小而是扩大，说明它的可扩展性更好；作者也指出，在更快的网络上（此时压缩开销在总成本里的占比更高），这一优势预计还会继续放大。

表 1 汇总了达到相同收敛目标（Cifar10 上 94% 精度、Wiki10-31K 上 80% 精度）时的端到端加速比。DRAGONN 相对 DGC 最高可达 3.52 $\times$ ，相对完整同步最高可达 35.9 $\times$ ，同时保持与完整同步一致的测试精度。

表 1 达到相同收敛精度时，DRAGONN 相对 DGC 与 FULL（完整同步）的端到端训练加速比（16 GPU）。

加速比基线	ResNet50	ViT	MLP-Mixer	XML
vs DGC	1.42 $\times$	2.15 $\times$	1.72 $\times$	3.52 $\times$
vs FULL	2.40 $\times$	2.45 $\times$	6.04 $\times$	35.9 $\times$

更快的前提是「不掉点」。图 4 把四个模型的收敛曲线画在一起：DRAGONN（绿色）的每一步收敛轨迹与完整同步（FULL，蓝色）以及 DGC、TopK 基本重合，达到同样精度所需的迭代数没有变差。这说明 DRAGONN 的提速来自压缩这一步更便宜，而不是靠牺牲收敛质量换来的。

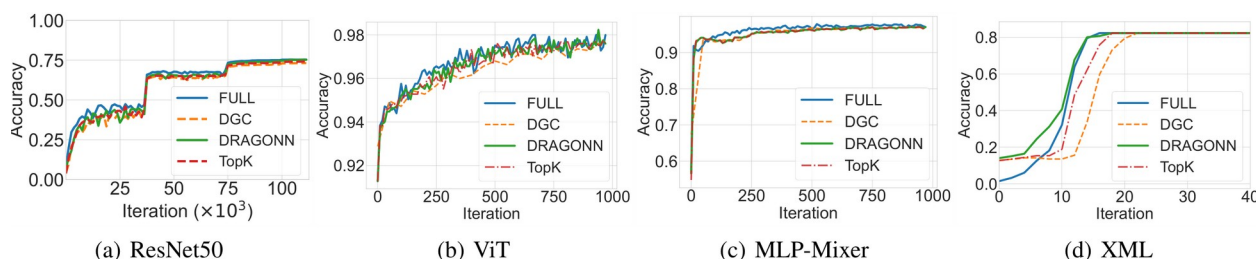


图 4 四个模型在不同稀疏化方法下的收敛对比（精度 vs 迭代数）。DRAGONN（绿色）与 FULL（蓝色）几乎重合，逐迭代收敛质量不受影响，提速并非以掉点为代价。

拆开看：三个组件各自贡献了多少

论文用一组微基准和消融实验来解释加速从何而来。图 5 左图对比了几种压缩算法的编码时间：在所有张量尺寸下，DRAGONN（绿色）的编码时间都最低，明显低于 DGC，更远低于精确 TopK；右图对比了稠密解码（Dense）与稀疏解码（Sparse），后者几乎是一条贴地的平线，而稠密解码随 GPU 增多线性上涨。

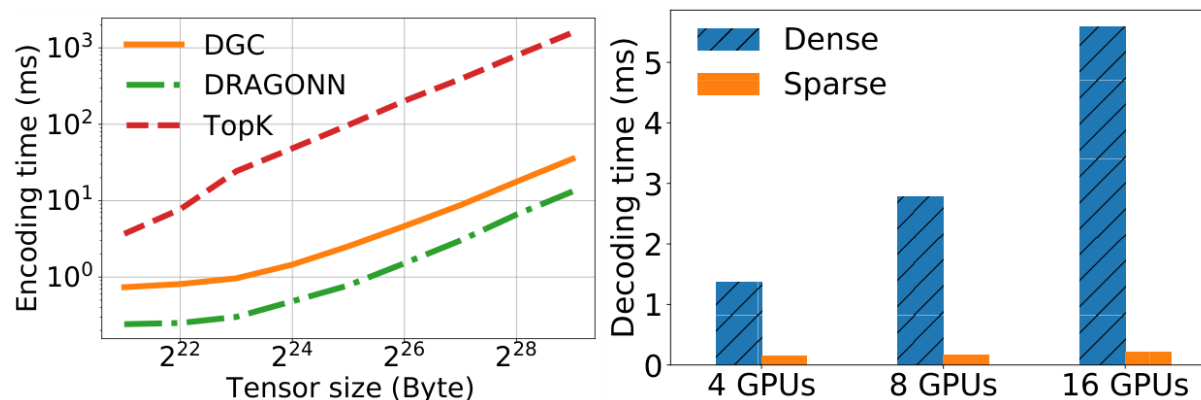


图 5 微基准。左：不同压缩算法在各张量尺寸下的编码时间（对数纵轴），DRAGONN 全程最低；右：稠密解码随 GPU 数量线性增长，而稀疏解码几乎恒定。

图 6 则逐个叠加三个组件，看它们对单步「计算」与「同步」时间的影响。仅哈希压缩器就带来基础提速；叠加效率感知张量选择后，在某个模型上提速可升到约 1.92 $\times$ ；再加上稀疏解码，最高可达 4.33 $\times$ 。有意思的是，稀疏解码带来的加速甚至超过 GPU 数量 K 倍——因为它同时消除了那份随 GPU 线性增长的解码计算成本。

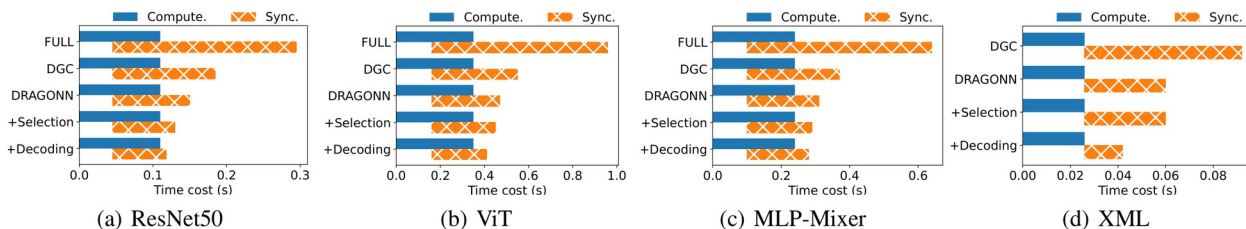


图 6 三个组件（哈希压缩器、效率感知张量选择 +Selection、稀疏解码 +Decoding）对单步计算（Compute.）与同步（Sync.）时间的逐步影响。每叠加一个组件，同步时间进一步下降，直到明显低于 DGC 与 FULL。

小结与边界

DRAGONN 的贡献可以浓缩成一个判断：既然梯度稀疏化本来就只需要近似的 top-k 梯度，那么用一个精确的并行前缀和去安放它们就是多余的开销。把它换成直接的随机哈希，一个原本串行、依赖繁重的压缩步骤就变成了可大规模并行的操作，压缩时间最多降低 70%；再配合效率感知张量选择与稀疏解码，就能在不损失精度的前提下拿到端到端可扩展的提速。

需要留意的是结论成立的边界。这些结果是在 16 张 V100、25Gbps 网络的特定测试平台上、针对上述四个模型得到的；哈希会引入少量冲突误差，论文依赖 memory-momentum 误差反馈来吸收它。作者也明确指出，DRAGONN 的相对优势与「通信 - 压缩」的比例有关——网络越快、张量越大，压缩越是瓶颈，它的收益也越大；反过来在通信极其昂贵而压缩本就不重的场景里，提升空间会相应变小。总体而言，这项工作把梯度稀疏化里一个长期被忽视的开销点摆到了台面上，并给出了一个足够简单、足够并行的解法。